

TD&TP3

EXERCICE1-TRI FUSION

procédure triFusion(indinf, indsup, tab)

Paramètres formels : indinf, indsup : Entier (E)
Variables intermédiaires : indmil : entier
tab : Tableau (E/S)

Début

```
Si indinf < indsup Alors
    indmil ← (indinf+indsup) / 2
    triFusion(indinf, indmil, tab)
    triFusion(indmil+1, indsup, tab)
    fusionner(indinf, indmil, indsup, tab)
Finsi
```

Fin

procédure Fusionner(indinf, indmil, indsup, tab)

Paramètres formels : indinf, indmil, indsup : entier (E)
Variables intermédiaires : tab1, tab2 : tableau
tab : tableau (E/S)
i, k, i1, i2 : entier

Début

```
{ Construction du tableau tab1 }
pour i variant de indinf à indmil faire
    tab1[i-indinf+1] ← tab[i]
finpour
{ Construction du tableau tab2 }
pour i variant de indmil+1 à indsup faire
    tab2[i-indmil] ← tab[i]
finpour
{Fusion des 2 tableaux triés}
tab1[indmil-indinf+1] ← VMAX
tab2[indsup-indmil+1] ← VMAX
i1 ← 1
i2 ← 1
Pour k variant de indinf à indsup Faire
    Si tab1[i1] < tab2[i2] Alors
        Tab[k] ← tab1[i1]
        I1 ← i1 + 1
    Sinon
        Tab[k] ← tab2[i2]
        I2 ← i2 + 1
    Finsi
Finpour
```

Pour k variant de indinf à indsup Faire

```
Si tab1[i1] < tab2[i2] Alors
    Tab[k] ← tab1[i1]
    I1 ← i1 + 1
Sinon
    Tab[k] ← tab2[i2]
    I2 ← i2 + 1
Finsi
```

Finpour

Fin

- 1) Illustrer le déroulement du tri par fusion sur le tableau d'entiers suivant [28, 25, 43, 19, 16, 32, 50, 10, 22].
- 2) Numéroté l'ordre des appels.

- 3) En ne comptabilisant que **les comparaisons entre les éléments du tableau**, trouver et analyser en fonction de **n** l'équation de la complexité en temps de l'**algorithme tri-Fusion** ci-dessus.

EXERCICE2 TRI-RAPIDE

procédure triRapide(indinf, indsup, tab)

Paramètres formels : indinf, indsup : entiers (E)
Variables intermédiaires : indpivot : entier
tab : tableau (E/S)

Début

```
Si indinf < indsup Alors
    Indpivot ← partitionner(indinf, indsup, tab)
    Tri_rapide(indinf, indpivot-1, tab)
    Tri_rapide(indpivot+1, indsup, tab)
Finsi
```

Fin

Fonction partitionner(indinf, indsup, tab) → Entier

Paramètres Formels indinf, indsup :Entier(E)
Variables Intermédiaires : l, k, indpivot : Rntier
tab : Tableau (E/S)
pivot : TElement

Début

```
Indpivot ← indinf
Pivot ← tab[indpivot]
L ← indinf + 1
K ← indsup
Tantque (l<=k) Faire
    tantque (l<=k) et (tab[l] <= pivot) faire
        L ← l + 1
    fintantque
    tantque (l<=k) et (tab[k]>pivot) faire
        K ← k - 1
    fintantque
    Si (l<k) Alors
        Permutation(tab[l], tab[k])
        L ← l + 1
        K ← k - 1
    Finsi
Fintantque
Si indpivot ≠ k alors
    Permutation(tab[indpivot], tab[k])
finsi
renvoyer k
```

Fin

```
Si indpivot ≠ k alors
    Permutation(tab[indpivot], tab[k])
finsi
renvoyer k
```

Fin

- 1) Illustrer le déroulement du tri rapide sur le tableau d'entiers suivant [28, 25, 43, 19, 16, 32, 50, 10, 22].
- 2) Numéroté l'ordre des appels.
- 3) En ne comptabilisant que **les comparaisons entre les éléments du tableau**, trouver et analyser en fonction de **n** l'équation de la complexité en temps de l'**algorithme tri-Rapide** ci-dessus.

EXERCICE 3 SUITE DE FIBONACCI

1) Récursivement, la suite de Fibonacci est définie par :

$$F(n) = F(n-1) + F(n-2) \text{ si } n \geq 2$$

$$F(n) = n \text{ si } n \leq 1$$

- Si l'on transcrit cette définition mathématique sous forme d'une fonction récursive, quelle en est la complexité (en nombre d'additions) ?
 - Ecrire un algorithme récursif qui calcule, pour $n > 0$, le couple $(F(n), F(n-1))$.
 - Utilisez l'algorithme précédent pour écrire un nouvel algorithme calculant $F(n)$.
 - Quelle est la complexité (en nombre d'additions) de cet algorithme ?
- 2) Le calcul par récurrence de cette suite consiste à transformer la récurrence d'ordre 2 en un système récurrent d'ordre 1, à l'aide de 3 suites d'entiers telles que :

$$F(i) = F_i = F_{i-1} + R_{i-1}$$

$$W_i = F_{i-1} \quad (\text{Suite technique de sauvegarde})$$

$$R_i = W_i \quad (\text{Suite retardée de } F_i : R_i = F_{i-1})$$

Avec $F_1 = 1$; $R_1 = F_0 = 0$.

Quelle est la complexité de cette solution itérative ?

- 3) Voici une solution originale abordant directement le problème sous l'angle des fonctions récursives due à David Gries : On essaie de supprimer l'un des deux appels récursifs à l'aide d'une fonction G définie par : $G(n) = \langle F(n), F(n-1) \rangle$ qui n'est définie que pour $n \geq 1$, avec :

$$G(1) = \langle 1, 0 \rangle \quad \text{si } n=1$$

$$G(n) = \langle F(n-1) + F(n-2), F(n-1) \rangle \quad \text{si } n \geq 2$$

Le second membre s'exprime uniquement en fonction des deux composantes de

$$G(n-1) = \langle F(n-1), F(n-2) \rangle.$$

Pour exprimer simplement ce lien entre $G(n)$ et $G(n-1)$, il est commode d'organiser $G(n)$ sous la forme :

$$G(n) = \begin{pmatrix} F(n) \\ F(n-1) \end{pmatrix} = \begin{pmatrix} F(n-1) + F(n-2) \\ F(n-1) \end{pmatrix}$$

- Transcrire cette formule en une relation matricielle de récurrence matricielle pour $G_n = G(n)$.
- En déduire la valeur de G_n .
- Calculer la complexité de l'algorithme correspondant.